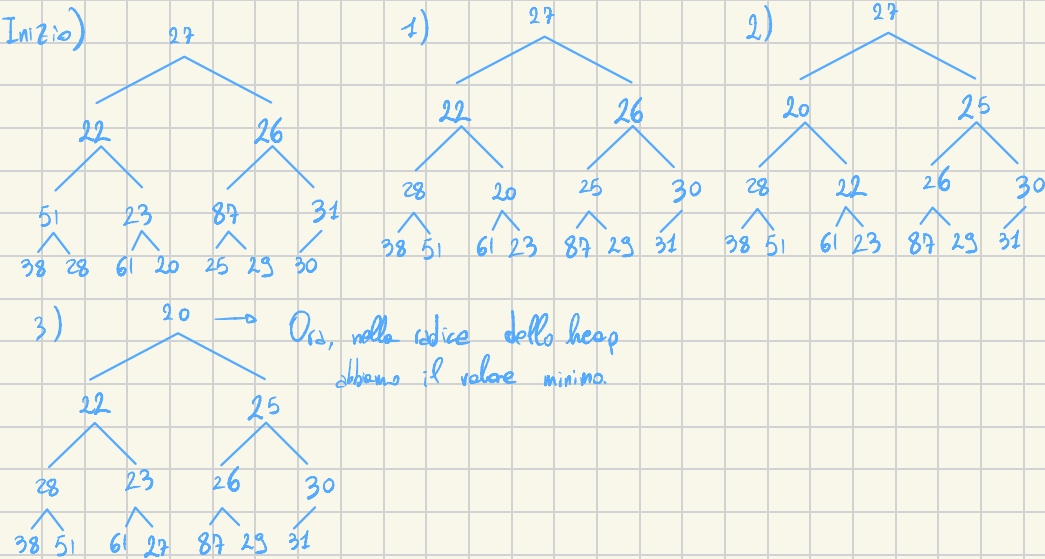


Es. 1)

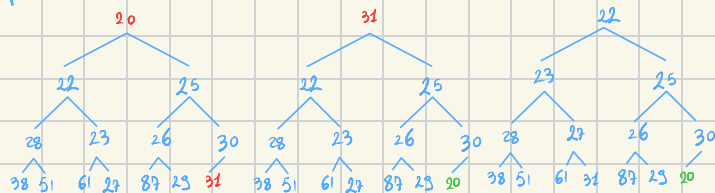
27 22 26 51 23 87 31 38 28 61 20 25 23 30
Trasformare in heap, con val minimo in radice. Eseguire poi i primi 2 passi di heap sort

>VOLGIMENTO:

27 | 22 26 | 51 23 87 31 | 38 28 61 20 25 23 30



Heap sort 1)

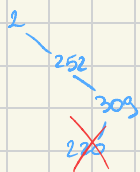


Es. 2) BST con num tra 0 e 400, cercare chiave 255. Quale corretta?

- a) 2 252 303 220 330 264 255
 b) 2 252 209 220 240 250 255
 c) 1 400 202 300 290 250 255

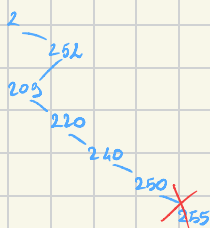
SVOLGIMENTO

a) 2 252 303 220 330 264 255



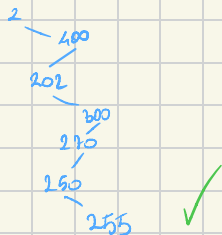
Errore: $220 < 303$ quindi ramo sx di 303, ma siamo già nel ramo dx di 252, ma $220 < 252$!

b) 2 252 209 220 240 250 255



Errore: 255 non può trovarsi nel ramo Sinistro di 250, ovvero fra gli elementi minori di 252.

c) 1 400 202 300 290 250 255



Es. 3) Tabella hash. Open addressing con linear probing. $\alpha < 0.4$

chiavi: 24 26 33 46 154 17 43 230 16

SVOLGIMENTO:

1) Dimensione tabella? Poiché $\alpha = \frac{N}{M} \Rightarrow M = \frac{N}{\alpha} = \frac{9}{0.4} = 22.5 \rightarrow$ Dare esse num primo $\Rightarrow M = 23$

2) Quale funzione? Escludiamo le opzioni a) e d) poiché quando abbiamo h_1 e h_2 stiamo usando double hashing, mentre qui si richiede linear probing con $M=23$, quindi opzione b).

3) SVOLGIMENTO

0	46 (230 bit!)	5		10	93	15		20	43
1	230	6		11		16	154 (16 bit!)	21	
2		7		12		17	17 (16 bit!)	22	
3	26	8		13		18	16		
4	211	9		14		19			

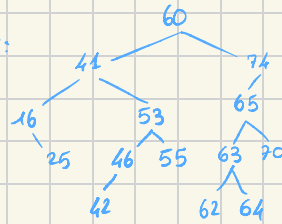
$$h(k) = (k \cdot i) \% M$$

Risposte alle domande:

1) Collisioni: 16: cella 1 = 16, cella 2 = 17

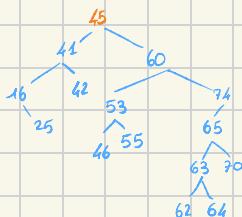
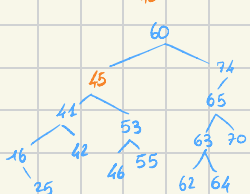
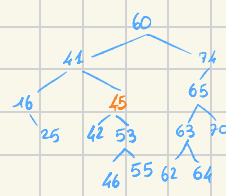
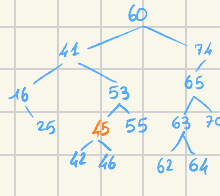
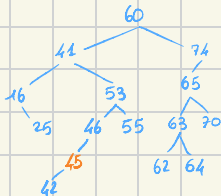
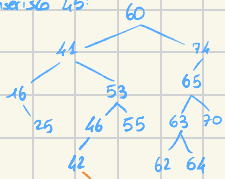
2) Posizione chiave 230? cella 1.

E> 4) Invertire in radice 45 poi cancellare 60. BST:



SVOLGIMENTO:

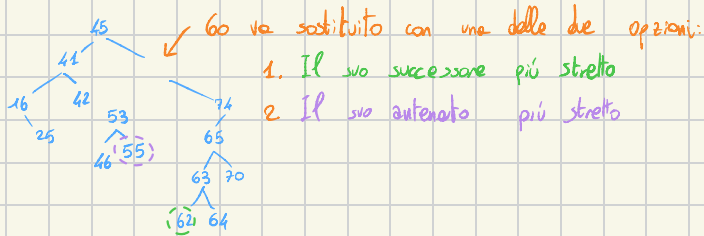
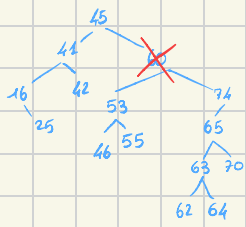
1) Inserisco 45:



RISPOSTE ALLE DOMANDE:

- figlio sx: 41
- figlio dx: 60

2) Cancellazione 60:

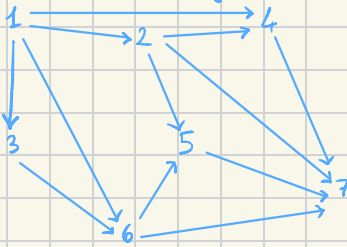


Successore più stretto

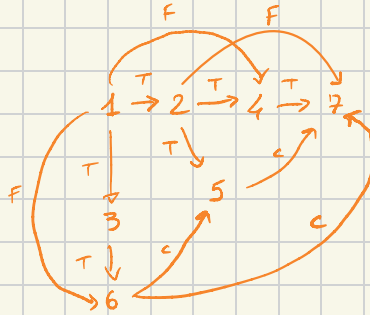
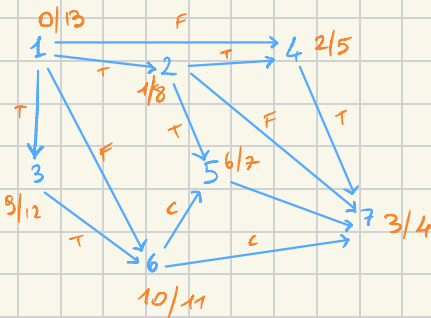
RISPOSTA ALLA DOMANDA:

- figlio sx 65: 63
- figlio dx 65: 70

Es. 5) Ordinare topologicamente DAG. 1 come partenza.

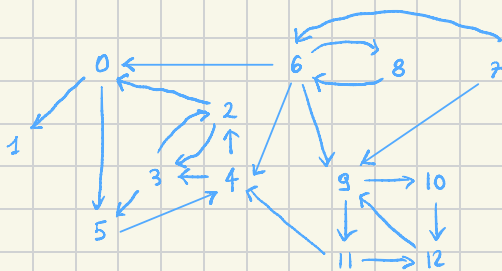


SVOLGIMENTO:



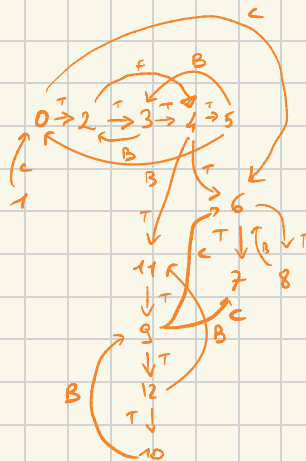
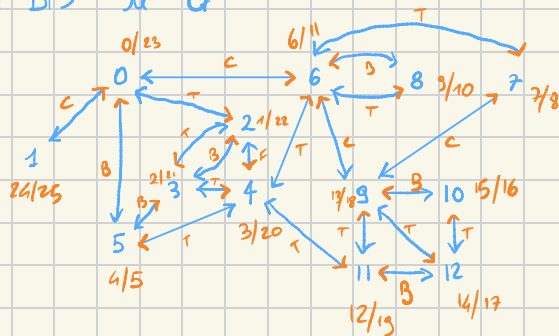
Ordinamento topologico (per tempo di terminazione decrescente): 1, 3, 6, 2, 5, 4, 7

Es 6) Algoritmo Kosaraju per SCC, partendo da 0.



SOLGIMENTO:

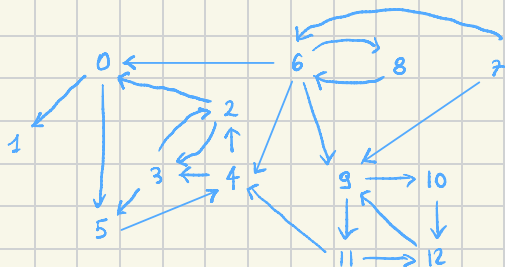
1) DFS su G^T



Ordine per t fine decrescente:

1, 0, 2, 3, 4, 11, 9, 12, 10, 6, 8, 7, 5

2) DFS su G con nuovo ordine.



1: 1

0: 0 → 5 → 4 → 2 → 3

11: 11 → 12 → 9 → 10

6: 6 → 8

7: 7